

Defender Repair Lab

Vibe Coding and AI-Assisted Software Engineering

Student	Atharva Anand
SRN	PES1UG24CS092
Course	Software Engineering
Date	7 October 2026
Implementation	Python 3 and Pygame

Submission links

[Updated Lab-4 repository](#) | [Complete Codex chat](#)

[Before video \(10 seconds\)](#) | [After video \(10 seconds\)](#)

Declaration

This submission was produced with OpenAI Codex as an AI pair-programming assistant. The report records the actual assistance, implementation, tests, and evidence. It does not present the work as unaided.

1. Assignment understanding

The supplied [Defender starter repository](#) contains one deliberate radar bug and three incomplete extension hooks. The game uses a 3,200-pixel wrapping world displayed through an 800-pixel player-centred viewport. The key engineering issue is keeping world-space data separate from screen-space drawing.

Deliverable	Completed result
Task 1	Absolute world-space radar mapping
Task 2	Capped wave-dependent sky color
Task 3	Timed +500 rescue popup
Task 4	Bonus life every 10,000 points
Verification	4 automated tests passed; bytecode compilation passed
Evidence	Two 10-second MP4 files and two genuine runtime frames
Version control	Starter import plus one separate commit per task

Design decisions

The changes are intentionally small and local. Existing game mechanics were preserved. Visual additions use the existing font and rendering loop, and the tests isolate the required behaviors without needing an interactive display.

2. Task implementation

Task 1: Fix radar world coordinates

Problem. The original radar called the player-centred `screen_x` method, so blips shifted with the camera.

Implementation. Each world x coordinate is normalized from the 0 to `WORLD_W` range into the 0 to `VIEW_W` radar range.

Verification. A test records the drawn blip rectangles and confirms x positions 800, 2400, and 1600 map to radar positions 200, 600, and 400.

```
rx = (x % WORLD_W) / WORLD_W * VIEW_W
```

Task 2: Implement wave sky color

Problem. `sky_color` returned no value, leaving the same background in every wave.

Implementation. A bounded wave level increases red, green, and blue components while retaining a dark, high-contrast background.

Verification. Tests confirm wave 1 uses (5, 5, 20), later waves differ, and every channel stays within 0 to 255.

```
level = min(max(int(wave) - 1, 0), 12)
return (5 + level * 2, 5 + level, 20 + level * 3)
```

2. Task implementation (continued)

Task 3: Add humanoid rescue feedback

Problem. The rescue hook was empty, so the 500-point reward had no visible confirmation.

Implementation. A 1.4-second timer is stored on the rescued Humanoid. The draw loop displays a floating green +500 RESCUE! label while the timer is active.

Verification. Tests confirm the timer starts at 1.4 seconds, decreases during update, and clamps to zero.

```
def on_humanoid_rescued(humanoid):  
    humanoid.rescue_timer = 1.4
```

Task 4: Set a bonus-life threshold

Problem. The hook returned no value, which disabled bonus lives even though bookkeeping already existed.

Implementation. The hook returns 10,000. Existing update logic awards one life whenever a new multiple is crossed.

Verification. The automated test asserts that the configured threshold is exactly 10,000.

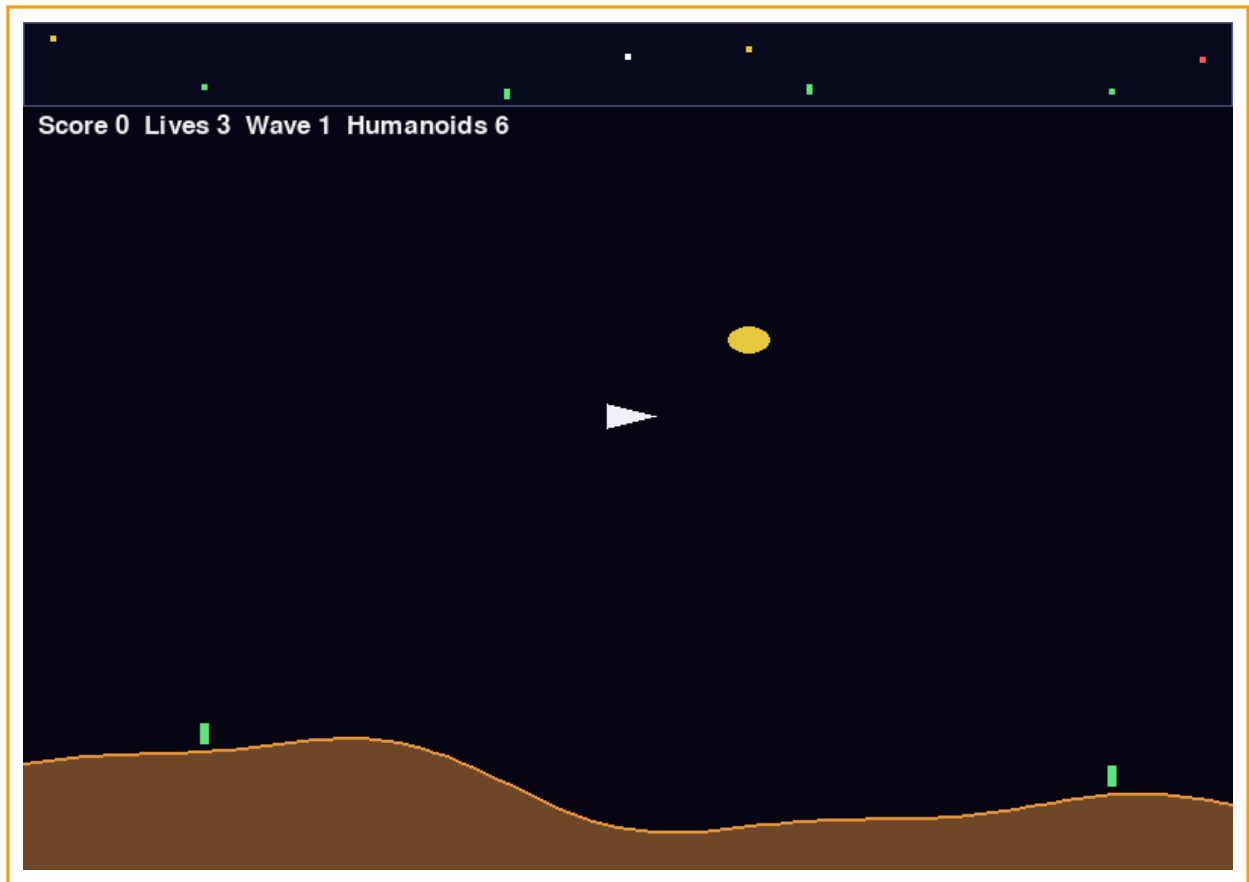
```
def bonus_life_threshold():  
    return 10_000
```

Behavior retained

Player and enemy wrapping, lander abduction, mutant conversion, bullet collisions, scoring, waves, lives, and game-over behavior remain unchanged.

3. Before evidence

Genuine runtime frame rendered from the unmodified starter. The radar uses player-relative viewport coordinates rather than absolute world positions. The linked 10-second video moves the player so the incorrect camera-dependent radar motion is visible.

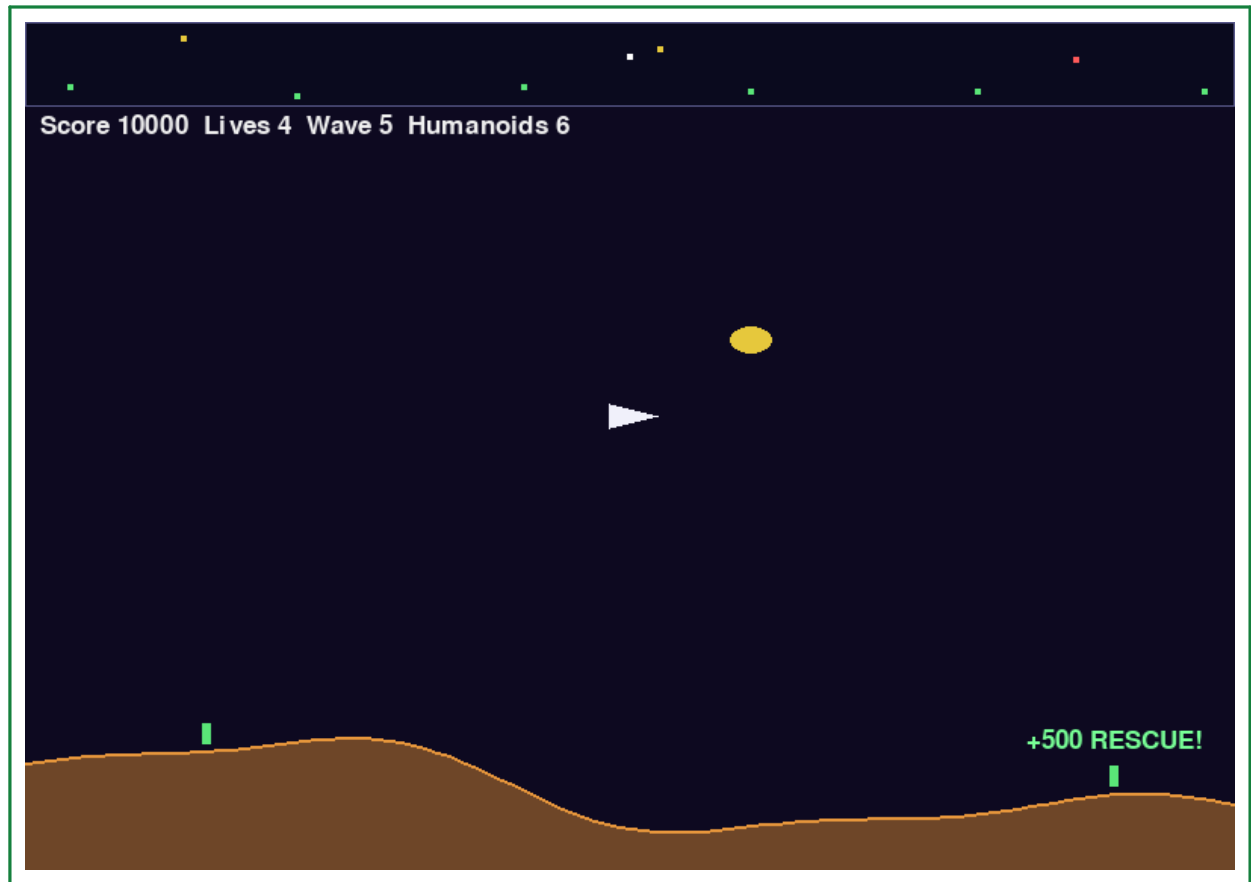


[Open before video](#)

Evidence provenance: the image and video were generated by importing and executing the original upstream game.py with deterministic entity positions. They are not mockups or reconstructed screenshots.

4. After evidence

Genuine runtime frame rendered from the completed Lab-4 code. Radar blips span the complete world, wave 5 uses the adjusted sky, the rescue popup is visible, and the HUD shows the 10,000-point bonus-life state.



[Open after video](#)

Evidence provenance: the image and video were generated by importing and executing the completed Lab-4 game.py. The 10-second recording shows the player moving while non-player radar blips remain at their true world positions.

5. Verification and version control

Automated verification

The final code was tested in a Python 3.12 virtual environment using Pygame 2.6.1 and pytest. Python bytecode compilation also completed successfully.

```
$ python -m pytest -q
.... [100%]
4 passed in 0.04s

$ python -m py_compile game.py test_game.py
completed with exit code 0
```

Commit history

Commit	Purpose
20baca3	Import original Defender starter
cd1db39	Task 1: fix radar world coordinates
9841ea6	Task 2: add wave-based sky colors
fe8478b	Task 3: show rescue reward feedback
4a09d36	Task 4: award bonus life every 10,000 points
01a7940	Add automated verification tests

Acceptance result

All four requested code tasks are implemented, the required task separation is visible in Git history, and the final repository contains executable code, tests, videos, still evidence, prompt evidence, and this report.

6. AI prompt evidence and submission checklist

Actual AI interaction

The initial request asked Codex to complete the SE vibe-coding assignment from the supplied Defender repository, advise when recording was required, prepare evidence, and publish the result to the student's SE GitHub repository. Codex then read the assignment, decomposed it into four tasks, implemented and reviewed each change, ran tests, generated evidence, and prepared this report.

The complete interaction is available at [the shared Codex chat](#). AI assistance is disclosed in this report and PROMPT_EVIDENCE.md.

Critical review performed

- Confirmed the radar uses absolute world position rather than the player-centred viewport.
- Checked that the sky-color progression is bounded and maintains contrast.
- Checked that rescue feedback expires and does not affect unrelated game state.
- Confirmed the bonus-life threshold and existing multiple-crossing bookkeeping.
- Ran automated tests, compilation, visual inspection, and Git-history checks.

Final submission checklist

Status	Required item	Evidence
DONE	Updated Lab repository link	Lab-4 on GitHub
DONE	PDF details and evidence	This document
DONE	Before video, 10 seconds	Open MP4
DONE	After video, 10 seconds	Open MP4
DONE	Prompts and chat history	Open complete chat
DONE	Separate task commits	Task 1 through Task 4 commits listed above